Image Processing in Python

Introduction to PyTorch

This is a tutorial.

Make sure you have your laptop at hand!

Python Tutorial

- We will only support Python 3.X

- Python tutorial from Google (a good starting point)

https://developers.google.com/edu/python/

- The official tutorial (TL;DR)

https://docs.python.org/3/tutorial/index.html

NumPy: Scientific Computing in Python

- Documentation

https://docs.scipy.org/doc/numpy/reference/index.html

- A quick tutorial

https://docs.scipy.org/doc/numpy/user/quickstart.html

- If you are switching from Matlab to Numpy

http://mathesaurus.sourceforge.net/matlab-numpy.html

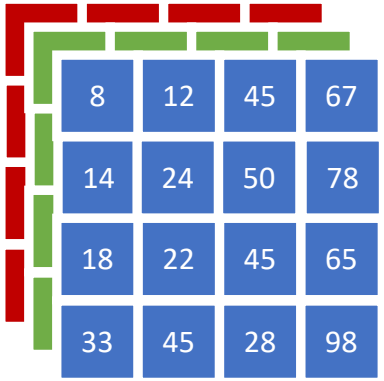
Image Processing Packages in Python

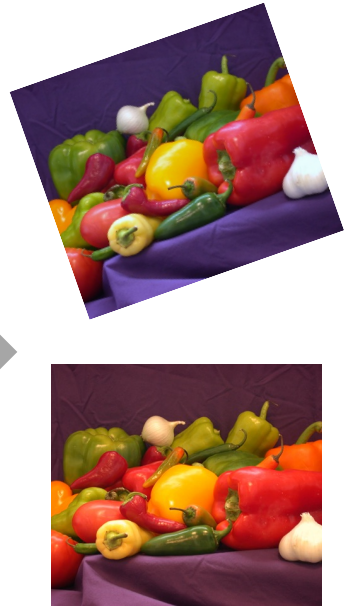
- Python Image Library – PIL (latest dev **Pillow**)
  - Light weighted library for basic image processing
  - Well polished functions
  - Limited functionality, not very efficient

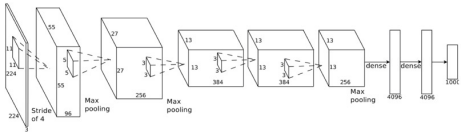
- Open-source Computer Vision (**OpenCV**)
  - Advanced library for image processing and computer vision
  - Good coverage of functionality, highly optimized
  - Sometimes can be buggy ☹

How Things Work …









Data IO

- Load images / videos
- Decode data
- Pillow/OpenCV

Image Transforms

- Pre-processing
- Data augmentation
- NumPy + Pillow/OpenCV

Learning Based Methods

- PyTorch
- TensorFlow

Goals

Basic image IO and image manipulations

How to use OpenCV and Pillow in Python

Image processing in Python

Resize, color transforms, filtering, rotation, …

Interface with PyTorch

When you need to load / process a lot of images

Before We Get Started

- Python 3.x installed on your laptop
  - Anaconda / Miniconda is recommended for installing Python
  - Conda is recommended for maintaining all packages
  - Virtual environment will be very helpful (using Python / Conda)

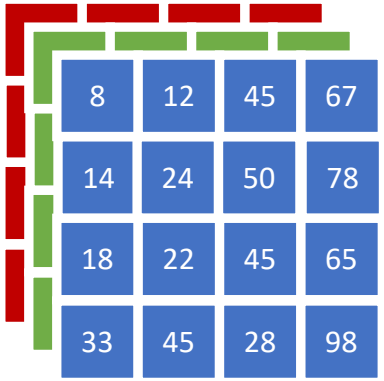
- We will install the following Python packages:
  - IPython / Notebook
  - Pillow
  - OpenCV
  - NumPy
  - PyTorch
  - Matplotlib

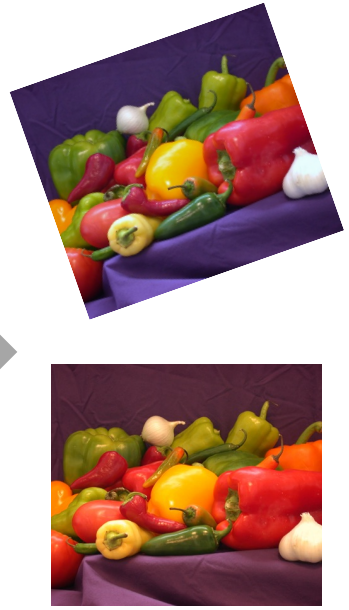
Installing Python Packages

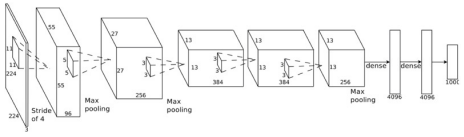
- Dependency and package management tools
- Conda vs PyPI
- *PyTorch (CPU version, see this link)*
  - *Windows: pip3 install torch torchvision*
  - *Linux: pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cpu*
  - *Mac: pip3 install torch torchvision*
- *All other dependencies*
  - *conda install ipython notebook numpy pillow opencv matplotlib*
  - If you have trouble with installing some of the packages, try pip install them

The Road Map









Learning Based Methods

Data IO

- Load images / videos
- Decode data
- Pillow/OpenCV

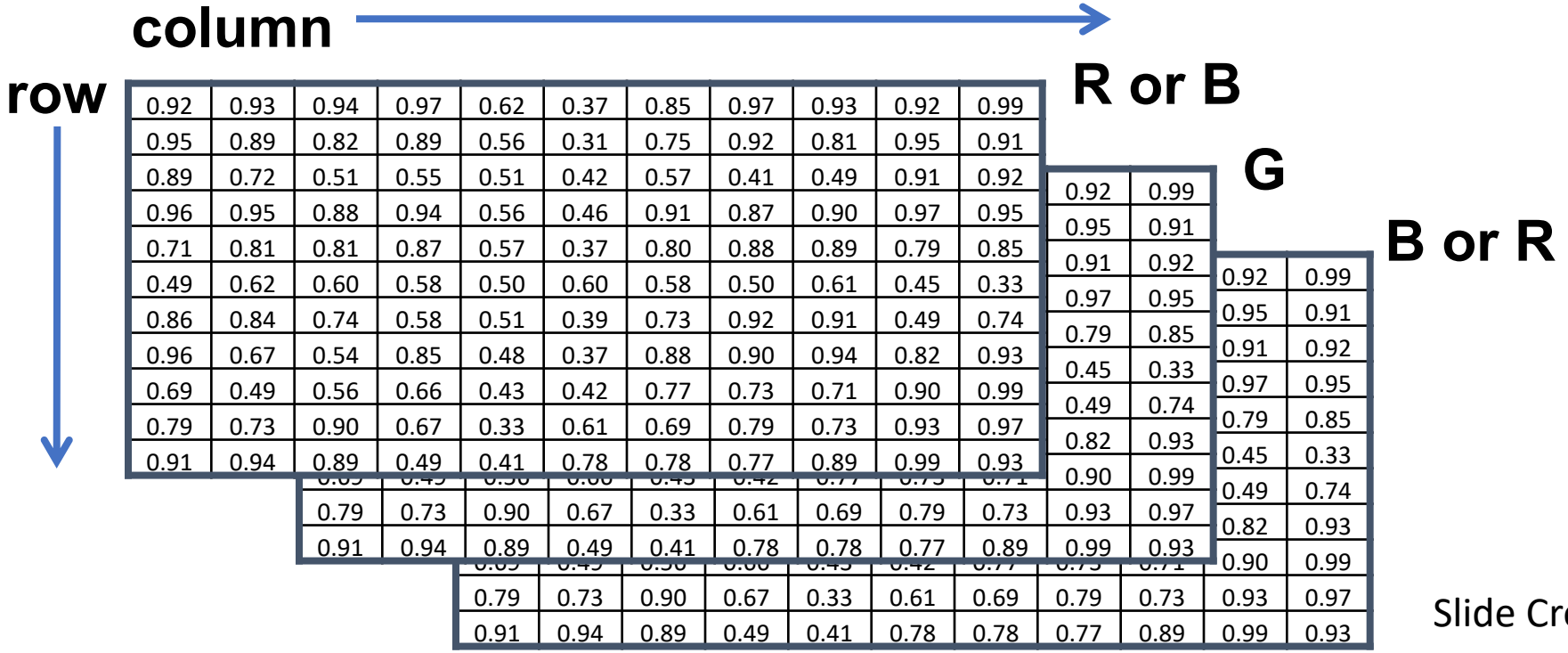
Image Transforms

- Pre-processing
- Data augmentation
- NumPy + Pillow/OpenCV

- PyTorch
- TensorFlow

Images in Python

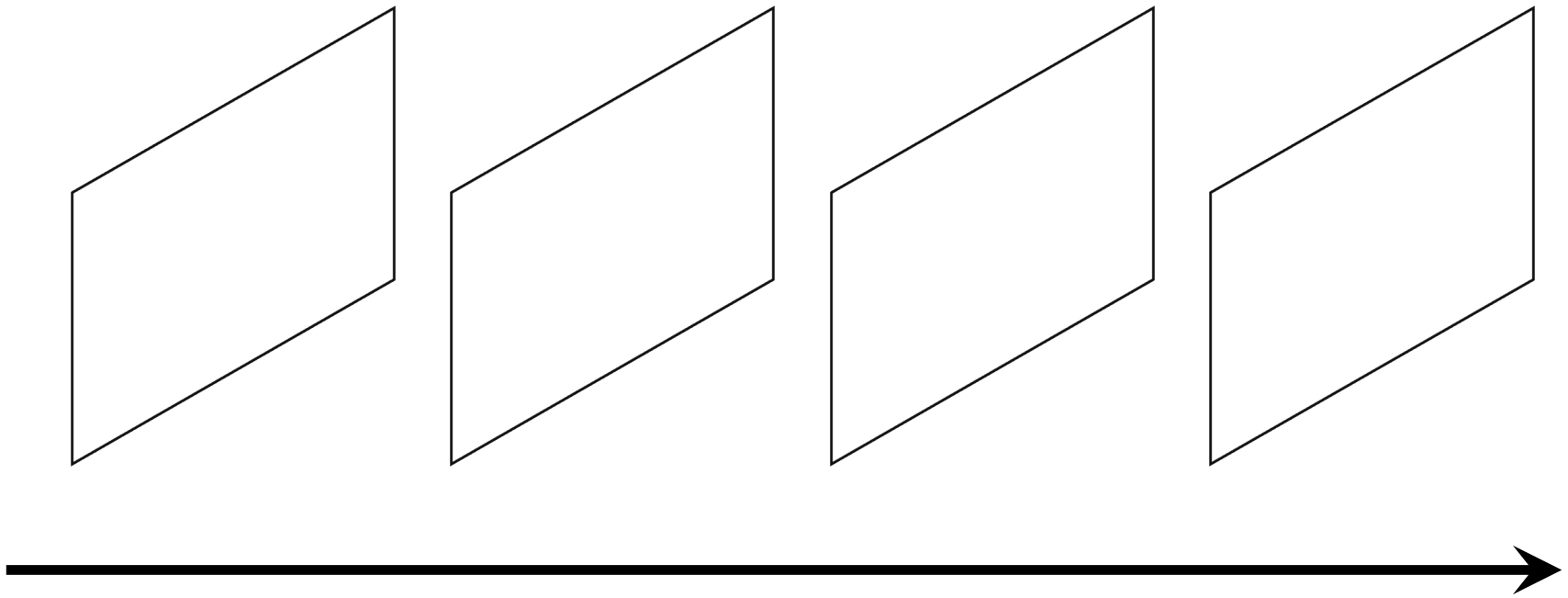
- Suppose we have a NxM RGB image called “im”
  - im(0,0,0) = top-left pixel value in 1st channel
  - im(y, x, b) = y pixels down, x pixels to right in the 2nd channel
  - im(N-1, M-1, 2) = bottom-right pixel in the 3rd channel



Slide Credit: James Hays

Videos in Python

- Stored as a sequence of images – known as video frames.
- Frame interval specified by the frame rate (e.g., frame per second)



Images / Videos: Sampling Continuous Signals

- Image as a mapping from coordinates to color values

$$\begin{pmatrix} c_r \\ c_g \\ c_b \end{pmatrix} = \boldsymbol{c} = f\begin{pmatrix} x \\ y \end{pmatrix} / f\begin{pmatrix} x \\ y \\ t \end{pmatrix}$$

  - f is continuous
  - Pixel values are sampled at discrete grids, i.e., (x_i, y_j)

Image IO (code on Canvas)

- Load / Save an image using PIL / OpenCV
- Convert the loaded image into NumPy array
- PIL and OpenCV loads a DIFFERENT channel ordering!
- Image resolution (W, H) & Data Type (uint8)

Image Manipulations (code on Canvas)

- Read a pixel value
- Modify pixel values within an image (careful: data type!)
- Swap the color channels
- Flip the image (vertical / horizontal)
- Crop a region from the image

Basic IO & Manipulation

```
from PIL import Image

im = Image.open("dog.jpg")
w, h = im.size
print('Original image size: %sx%s' % (w, h))
print('Image mode is: %s' % im.mode)
```

```
Original image size: 313x161
Image mode is: RGB
```



```
out_hf = im.transpose(Image.FLIP_LEFT_RIGHT)
out_vf = im.transpose(Image.FLIP_TOP_BOTTOM)
out_hf.save("dog_hf.jpg")
out_vf.save("dog_vf.jpg")
box = (0, 0, 100, 100)
region = im.crop(box)
region.save("dog_crop.jpg")
```





```
import numpy as np

region_np = np.array(region)
print('Data type is:', region_np.dtype)
w, h, c = region_np.shape
print('Original image size: %sx%sx%s' % (w, h, c))
region_np[0:20, 0:20] = np.array([0, 0, 255])
region_new = Image.fromarray(np.uint8(region_np))
region_new.save("dog_crop_new.jpg")
```

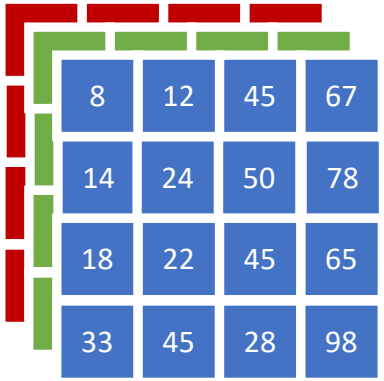
```
Data type is: uint8
Original image size: 100x100x3
```

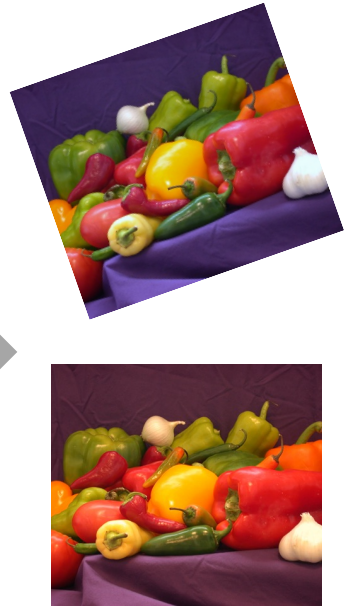


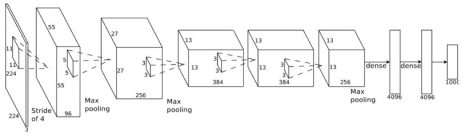


The Road Map









Learning Based Methods

Data IO

- Load images / videos
- Decode data
- Pillow/OpenCV

Image Transforms

- Pre-processing
- Data augmentation
- NumPy + Pillow/OpenCV

- PyTorch
- TensorFlow

Image Resizing (Zoom in / out)

- Increase / reduce the resolution of the image
- Different interpolation schemes (how to fill in the pixels)
- Sometimes can be tricky

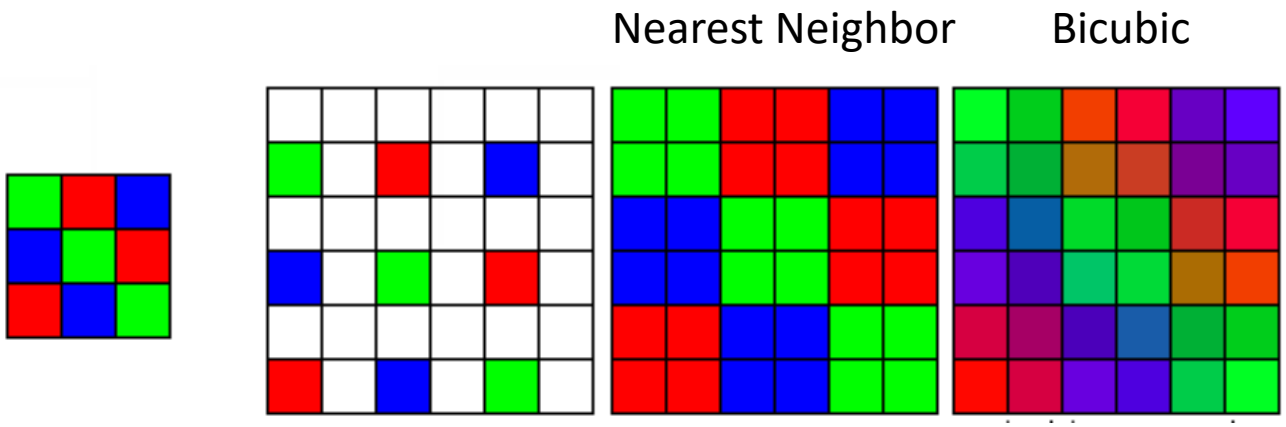
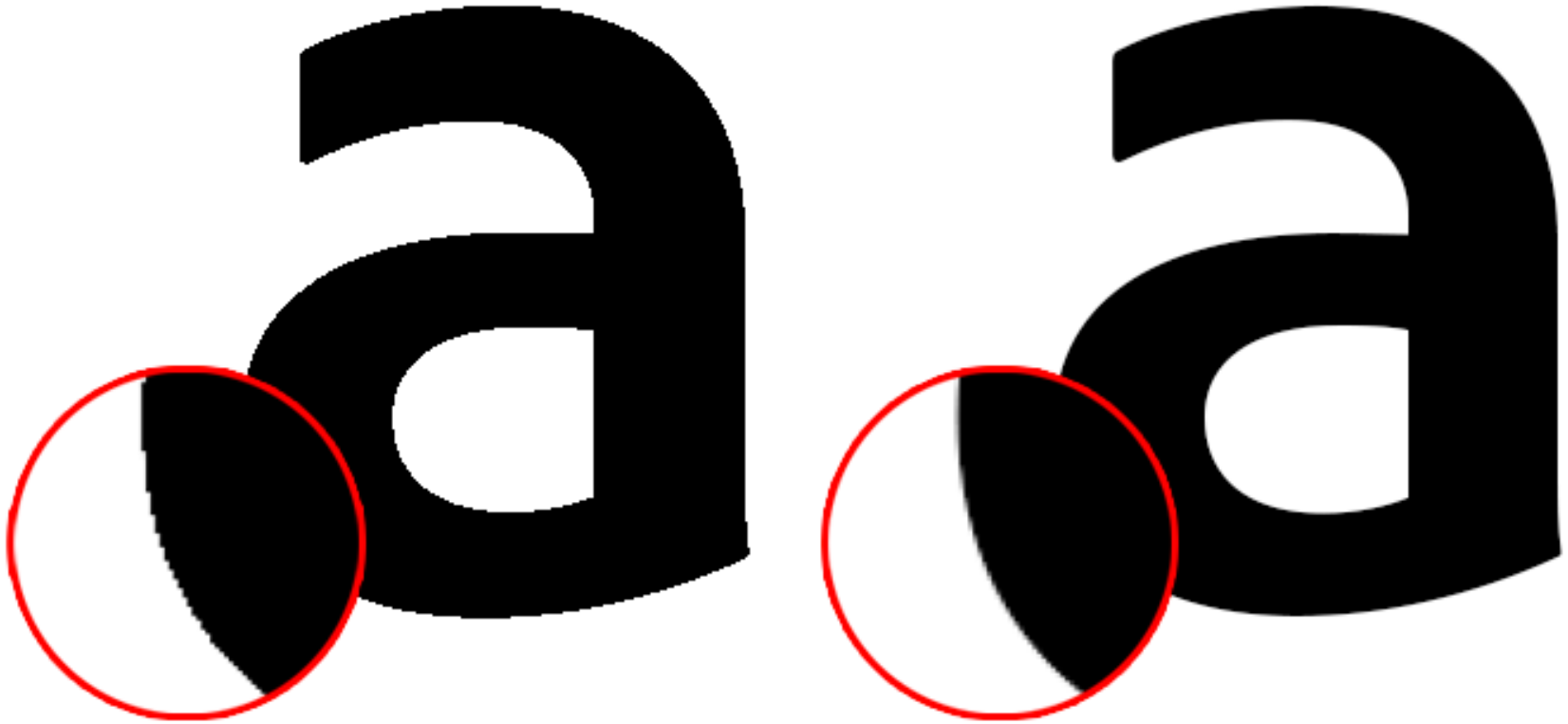


Image Resizing

- Anti-aliasing (blur a bit before resizing)

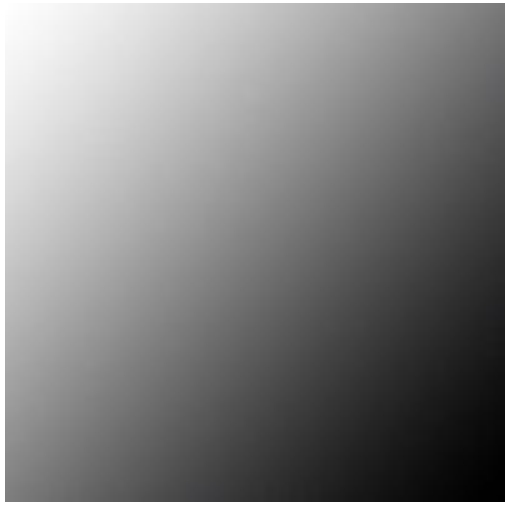


What if you want to resize a mask?



Pitfall of nearest neighbor interpolation

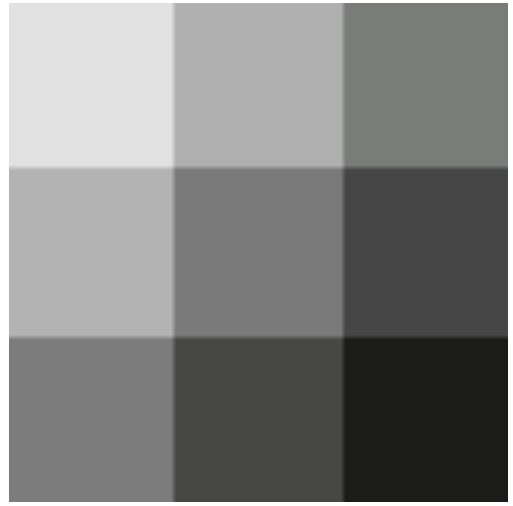
Input Image



OpenCV



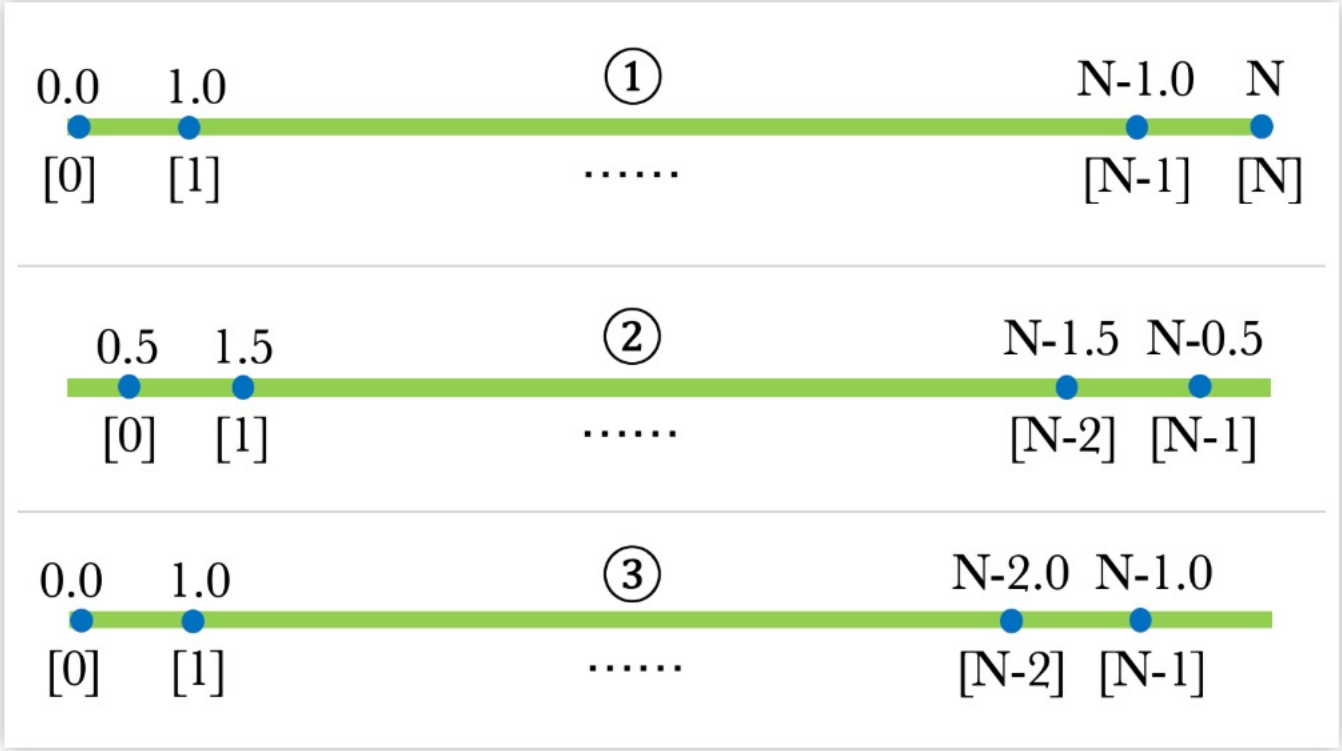
Pillow



https://github.com/opencv/opencv/issues/9096

Pitfall of nearest neighbor interpolation

- Recall that pixel values are sampled from a continuous signal
- But where are pixels located?
- Consider 1D case



Some further reading:
https://ppwwyyxx.com/blog/2021/Where-are-Pixels/

Image Upsampling / Downsampling

```
[ ]: # Part II: Image Resampling
im = cv2.imread("./samples/peppers.jpg")
h, w, c = im.shape
print('Original image size: %sx%s' % (w, h))
up_factor = 2
im_up_nn = cv2.resize(im, (up_factor*w, up_factor*h), interpolation = cv2.INTER_NEAREST)
im_up_bl = cv2.resize(im, (up_factor*w, up_factor*h), interpolation = cv2.INTER_LINEAR)
im_up_bc = cv2.resize(im, (up_factor*w, up_factor*h), interpolation = cv2.INTER_CUBIC)
h, w, c = im_up_nn.shape
print('Upsampled image size: %sx%s' % (w, h))
cv2.imwrite("./outputs/peppers_up_nn.jpg", im_up_nn)
cv2.imwrite("./outputs/peppers_up_bl.jpg", im_up_bl)
cv2.imwrite("./outputs/peppers_up_bc.jpg", im_up_bc)

import IPython.display
listOfImageNames = ['./outputs/peppers_up_nn.jpg',
                    './outputs/peppers_up_bl.jpg',
                   './outputs/peppers_up_bc.jpg']

for imageName in listOfImageNames:
    IPython.display.display(IPython.display.Image(filename=imageName))
```

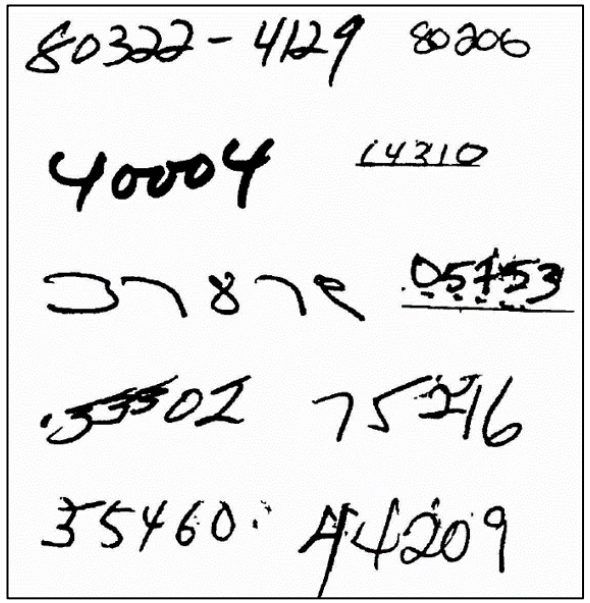
```
[ ]: im = cv2.imread("./samples/peppers.jpg")
h, w, c = im.shape
print('Original image size: %sx%s' % (w, h))
down_factor = 2
im_down_nn = cv2.resize(im, (w//down_factor, h//down_factor), interpolation = cv2.INTER_NEAREST)
im_down_bl = cv2.resize(im, (w//down_factor, h//down_factor), interpolation = cv2.INTER_LINEAR)
im_down_bc = cv2.resize(im, (w//down_factor, h//down_factor), interpolation = cv2.INTER_CUBIC)
h, w, c = im_down_nn.shape
print('Downsampled image size: %sx%s' % (w, h))
cv2.imwrite("./outputs/peppers_down_nn.jpg", im_down_nn)
cv2.imwrite("./outputs/peppers_down_bl.jpg", im_down_bl)
cv2.imwrite("./outputs/peppers_down_bc.jpg", im_down_bc)
```





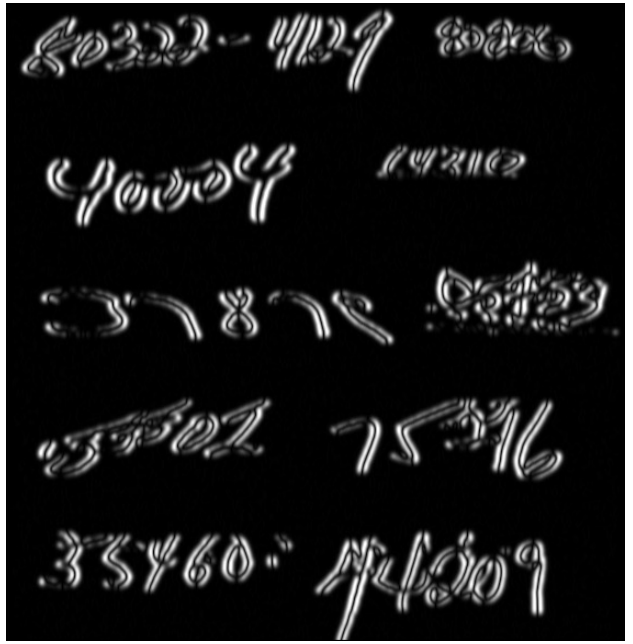
Image Filtering

Input



-1	0	+1
-2	0	+2
-1	0	+1

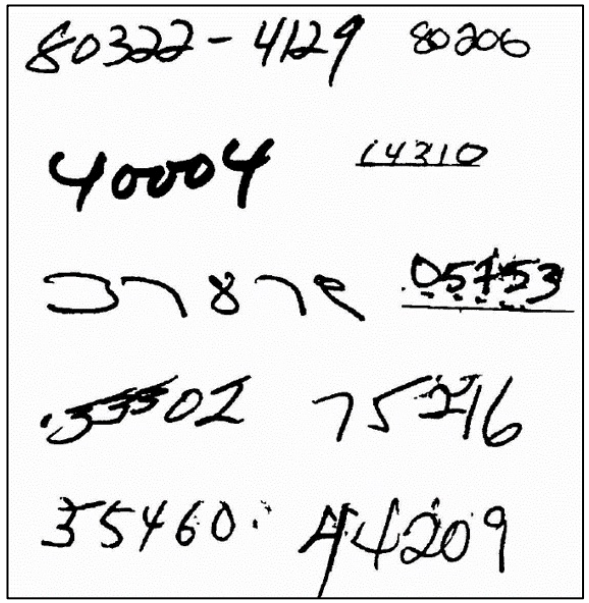
x filter



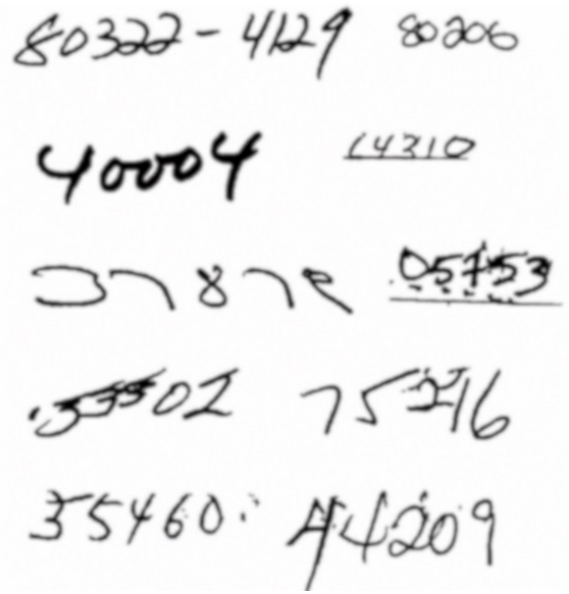
Edge Detection

Image Filtering

Input

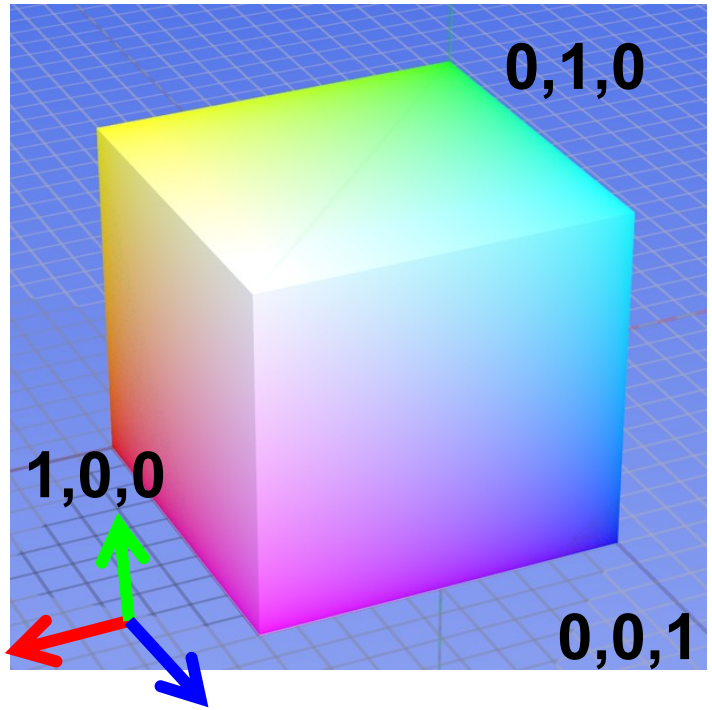


$$\frac{1}{16} \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



Blur

Image Color Space: RGB



Some drawbacks

- Strongly correlated channels
- Non-perceptual





R
(G=0,B=0)



G
(R=0,B=0)



B
(R=0,G=0)

Slide Credit: James Hays

Image from: http://en.wikipedia.org/wiki/File:RGB_color_solid_cube.png

Image Color Perturbation

- Rescale the color channels a bit (color jittering)
- Look quite different yet both can be realistic





Image Rotation & Warping: Examples



translation



rotation



aspect



affine

Image Rotation & Warping

- Affine Transform

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \text{Affine}\begin{pmatrix} x \\ y \end{pmatrix} = A\begin{pmatrix} x \\ y \end{pmatrix} + \text{b}$$

  - Translation: A = I
  - Cropping: applying a Translation + clipping the coordinates
  - Rotation: A = R (rotation matrix), b=0
  - Aspect (Resize): A = S (diagonal matrix)
  - More general: affine transformation
  - **Note: An affine transformation is a *linear and invertible* operation defined on the coordinates**

Image Rotation & Warping

- *How to compute pixels values after an affine transform?*
- Recall that an image is a mapping from 2D coordinates to color values

$$\boldsymbol{c} = f\begin{pmatrix} x \\ y \end{pmatrix}$$

- After an affine transform, the new image is now

$$\boldsymbol{c} = f'\begin{pmatrix} x \\ y \end{pmatrix}$$

Map the coordinates back into the original image plane

Retrieve the original color values (interpolation needed if not on integer grids)

$$= \underline{f}\left(\underline{\text{Affine}^{-1}\begin{pmatrix} x \\ y \end{pmatrix}}\right)$$

Image Rotation & Warping

- *How to compute pixels values after an affine transform?*
- Recall that an image is a mapping from 2D coordinates to color values

$$\boldsymbol{c} = f\begin{pmatrix} x \\ y \end{pmatrix}$$

- After an affine transform, the new image is now

$$\boldsymbol{c} = f'\begin{pmatrix} x \\ y \end{pmatrix}$$

This is called backward warping and can be used to compute all pixel values defined on 2D grids

$$= f\left(\text{Affine}^{-1}\begin{pmatrix} x \\ y \end{pmatrix}\right)$$

Image Rotation & Warping: Examples



translation



rotation



aspect



affine



perspective



cylindrical

Image Processing Examples

```python
# Part II: Image Transforms
import numpy as np
im = cv2.imread("./samples/peppers.jpg")
h, w, c = im.shape
print('Original image size: %sx%s' % (w, h))

im_gaussian = cv2.GaussianBlur(im, (9, 9), 0)
image_center = tuple(np.array(im.shape[1::-1]) / 2)
rot_mat = cv2.getRotationMatrix2D(image_center, 30, 1.0)
im_rotated = cv2.warpAffine(im, rot_mat, im.shape[1::-1], flags=cv2.INTER_LINEAR)
im_float = im.astype(np.float32)
im_float[:, :, 2] = np.clip(im_float[:, :, 2] * 0.6, 0, 255)
im_jittered = im_float.astype(np.uint8)
cv2.imwrite("./outputs/peppers_gaussian.jpg", im_gaussian)
cv2.imwrite("./outputs/peppers_rotated_0.jpg", im_rotated)
cv2.imwrite("./outputs/peppers_jittered.jpg", im_jittered)

import IPython.display
listOfImageNames = ['./outputs/peppers_gaussian.jpg',
                    './outputs/peppers_rotated_0.jpg',
                    './outputs/peppers_jittered.jpg']

for imageName in listOfImageNames:
    IPython.display.display(IPython.display.Image(filename=imageName))
```







See also the Pillow version

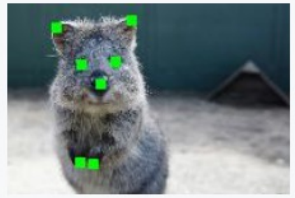
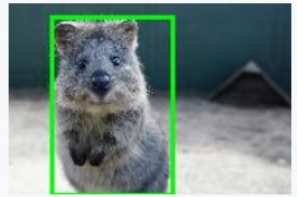
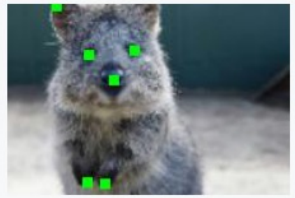
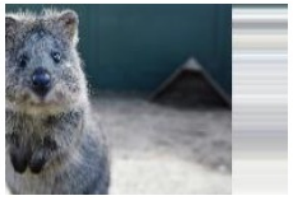
Multiple (Random) Image Transforms

- Flip horizontally
- Color jittering
- Rotation



https://colab.research.google.com/drive/109vu3F1LTzD1gdVV6cho9fKGx7lzbFll#scrollTo=BM7-0e4pzLXN

Multiple Image Transforms + Label Transforms

	Image	Heatmaps	Seg. Maps	Keypoints	Bounding Boxes
Original Input					
Gauss. Noise + Contrast + Sharpen					
Affine					
Crop + Pad					

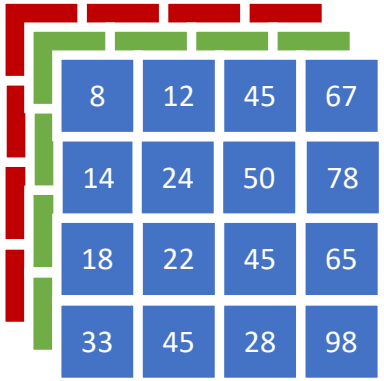
https://github.com/aleju/imgaug

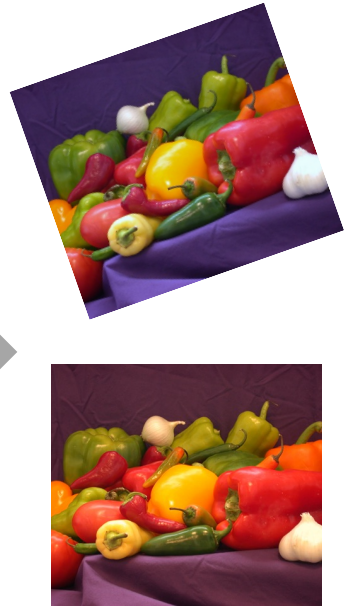
Multiple Image Transforms as **Data Augmentation**

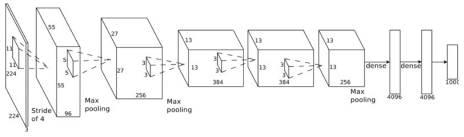
- Image transforms can drastically change the pixel values
- But the semantics often remain the same
- Get multiple different versions from the same data sample
- Help the learning, e.g., by preventing overfitting

The Road Map









Learning Based Methods

Data IO

- Load images / videos
- Decode data
- Pillow/OpenCV

Image Transforms

- Pre-processing
- Data augmentation
- NumPy + Pillow/OpenCV

- PyTorch
- TensorFlow

A Toy Problem: Cat vs. Dog











Data Source: https://www.kaggle.com/c/dogs-vs-cats

Loading and Processing Images in PyTorch

- A multi-process data loader that is highly flexible
  - torch.utils.data.DataLoader

- Support data loading and data processing (e.g., augmentation)

- Full documentation here

PyTorch Dataloader: A Simple Dataset

```python
# a simple dataset
class SimpleDataset(data.Dataset):
    """
    A simple dataset using PyTorch dataloader
    """
    def __init__(self, root_folder, file_ext, transforms=None):
        # root folder, split
        self.root_folder = root_folder
        self.transforms = transforms
        self.file_ext = file_ext

        # list all files
        file_list = glob.glob(os.path.join(root_folder, '*.{:s}'.format(file_ext)))
        self.file_list = file_list

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, index):
        # load img and label (from file name)
        filename = self.file_list[index]
        img = load_image(filename).astype(np.float32) / 255.0
        label = 1 if "cat" in os.path.basename(filename) else 0
        # apply data augmentation
        if self.transforms is not None:
            img  = self.transforms(img)
        return img, label
```

PyTorch Dataloader: Data Augmentation

```python
# simple data augmentation
class Compose(object):
    """
    Composes several transforms together.
    Args:
      transforms (list of ``Transform`` objects): list of transforms to compose.
    Example:
      >>> Compose([
      >>>     Scale(320),
      >>>     RandomSizedCrop(224),
      >>> ])
    """
    def __init__(self, transforms):
        self.transforms = transforms

    def __call__(self, img):
        for t in self.transforms:
            img = t(img)
        return img

    def __repr__(self):
        repr_str = ""
        for t in self.transforms:
            repr_str += t.__repr__() + '\n'
        return repr_str

class ToTensor(object):
    """Convert a ``numpy.ndarray`` image to tensor.
    Converts a numpy.ndarray (H x W x C) image in the range
    [0, 255] to a torch.FloatTensor of shape (C x H x W) in the range [0.0, 1.0].
    """
    def __call__(self, img):
        assert isinstance(img, np.ndarray)
        # convert image to tensor
        assert (img.ndim > 1) and (img.ndim <= 3)
```

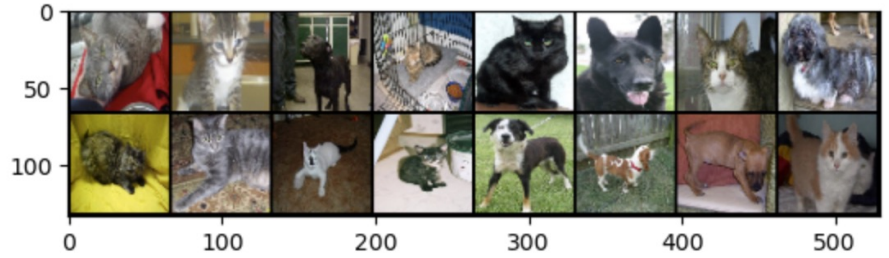
PyTorch Dataloader: Putting things Together

```
# image IO using PyTorch dataloader
# set up the transforms (make sure you convert numpy array to pytorch tensor)
transforms = []
transforms.append(Resize(64))
transforms.append(ToTensor())
comp_transforms = Compose(transforms)

# let us try a toy dataset
batch_size = 16
train_dataset = SimpleDataset('./data/mini_train', file_ext='jpg', transforms=comp_transforms)
val_dataset = SimpleDataset('./data/mini_val', file_ext='jpg', transforms=comp_transforms)
train_data_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_data_loader = torch.utils.data.DataLoader(val_dataset, batch_size=4, shuffle=False)

# get some random training images
dataiter = iter(train_data_loader)
images, labels = next(dataiter)

# show images
imshow(torchvision.utils.make_grid(images))
batch_size = 16
classes = ("dog", "cat")
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))
del dataiter
```




```
cat   cat   dog   cat   cat   dog   cat   dog   cat   cat   cat   cat   dog   dog   dog   cat
```

A Simple Neural Network for Cat vs. Dog

```
# a simple neural network modified from PyTorch tutorial
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 8, 5, padding=2, stride=2)
        self.bn1 = nn.BatchNorm2d(8)
        self.conv2 = nn.Conv2d(8, 16, 5)
        self.bn2 = nn.BatchNorm2d(16)
        self.conv3 = nn.Conv2d(16, 32, 5)
        self.bn3 = nn.BatchNorm2d(32)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 5 * 5, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 2)

    def forward(self, x):
        x = F.relu(self.bn1(self.conv1(x)))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = self.pool(F.relu(self.bn3(self.conv3(x))))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```

Training Neural Network for Cat vs. Dog

```
# loss function for training
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(net.parameters(), lr=0.001)

# training loop
for epoch in range(6):  # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(train_data_loader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 50 == 49:    # print every 50 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 50:.3f}')
            running_loss = 0.0

print('Finished Training')
```

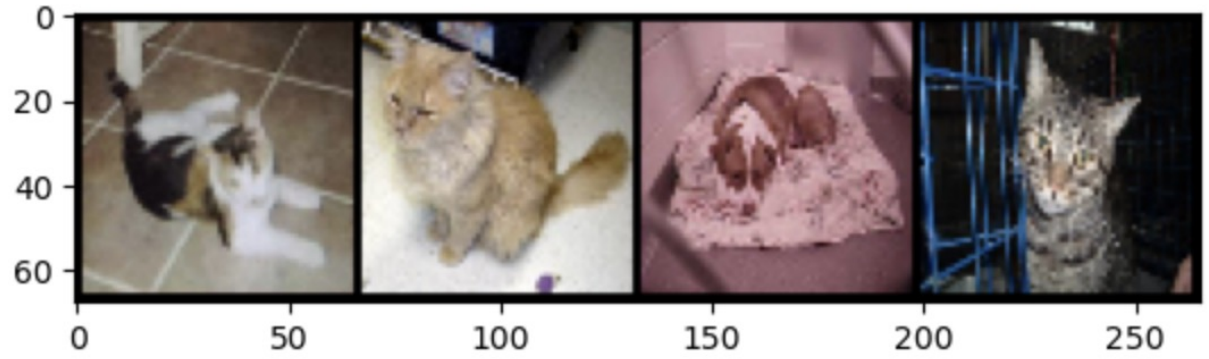
Evaluating Neural Network for Cat vs. Dog

```
# let us look at network outputs using the validation set
dataiter = iter(val_data_loader)
images, labels = next(dataiter)

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join(f'{classes[labels[j]]:5s}' for j in range(4)))

# prediction using the trained network
outputs = net(images)
_, predicted = torch.max(outputs, 1)

print('Predicted  : ', ' '.join(f'{classes[predicted[j]]:5s}'
                                for j in range(4)))
del dataiter
```




```
GroundTruth:  cat   cat   dog   cat
Predicted  :  cat   cat   dog   cat
```

- Basic image IO and manipulations
- Simple image transforms
- Loading and processing images in PyTorch
- A toy neural network in PyTorch

The end